

A certified outer bound for convex NLP scenario subproblems

mpi-sppy

companion to `doc/designs/ipopt_outer_bound_design.md`

Abstract

Progressive hedging needs an outer bound on the optimal value. The usual device is to read the dual bound off the subproblem solver, which fails for Ipopt: Ipopt is not a branch-and-bound solver and reports no dual bound. This note derives the bound that the `ipopt_outer_bound` spoke computes instead. The construction is Lagrangian weak duality combined with a tangent-plane underestimator minimised in closed form over the variable box. Its defining property is that it requires *no* assumption about the accuracy of the solve: a truncated or sloppy solve yields a loose bound rather than an invalid one. Convexity, by contrast, is genuinely load-bearing, and Section 5 isolates the one place where it is easy to get backwards.

1 Setting

Fix a scenario s with probability p_s , $\sum_s p_s = 1$. Let $v \in \mathbb{R}^n$ collect *all* variables of the scenario subproblem — the nonanticipative variables $x(v)$ together with the recourse variables — and let

$$B = \prod_{i=1}^n [l_i, u_i]$$

be the box of variable bounds. With hub weights W_s , the subproblem the spoke solves is

$$L_s(W_s) = \min_v \{ f_s(v) + W_s^\top x(v) : g_s(v) \leq 0, h_s(v) = 0, v \in B \}, \quad (1)$$

with $f_s : \mathbb{R}^n \rightarrow \mathbb{R}$, $g_s : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $h_s : \mathbb{R}^n \rightarrow \mathbb{R}^k$. This is exactly the problem the ordinary Lagrangian spoke builds, with the proximal term switched off.

Why the returned objective value is useless here. For a minimisation, the value the solver reports is the objective evaluated at the point it stopped at. Any such value is $\geq$ the minimum, so it is an *inner* bound: the wrong direction. Nor does Ipopt's convergence tolerance close the gap, since it is a relative KKT-residual tolerance rather than a bound on the objective error, and the two error sources point opposite ways — optimality error stops slightly above the optimum, while constraint violation can place the iterate slightly below it.

2 Weak duality

Lemma 1 (Weak duality). *Let $\lambda \in \mathbb{R}_+^m$ and $\mu \in \mathbb{R}^k$ be arbitrary. Define*

$$\varphi_s(v) = f_s(v) + W_s^\top x(v) + \lambda^\top g_s(v) + \mu^\top h_s(v), \quad (2)$$

$$q_s(\lambda, \mu) = \inf_{v \in B} \varphi_s(v). \quad (3)$$

Then $q_s(\lambda, \mu) \leq L_s(W_s)$.

Proof. Let v be any point feasible for (1). Then $v \in B$, so $q_s(\lambda, \mu) \leq \varphi_s(v)$. Moreover $g_s(v) \leq 0$ and $\lambda \geq 0$ give $\lambda^\top g_s(v) \leq 0$, and $h_s(v) = 0$ gives $\mu^\top h_s(v) = 0$; hence $\varphi_s(v) \leq f_s(v) + W_s^\top x(v)$. Chaining the two inequalities, $q_s(\lambda, \mu) \leq f_s(v) + W_s^\top x(v)$ for every feasible v . Taking the infimum over feasible v gives the claim. $\square$

Note what Lemma 1 does *not* require: λ and μ need not be optimal, or even good. Any $\lambda \geq 0$ and any μ will do. This is the property that makes the whole approach robust, and it is why a mistaken sign convention or a stale multiplier can only cost tightness (Remark 4).

The trap. Lemma 1 is not yet usable, because q_s is defined by an *infimum*. Handing φ_s to a nonlinear solver returns a *point*, and the value there is $\geq$ the infimum — an upper bound on q_s , which is the wrong direction again. A second NLP solve therefore does not by itself certify anything.

3 The box underestimator

Theorem 2 (Certified bound). *Suppose φ_s is convex on an open set containing B and differentiable at $\hat{v}$. Define*

$$\hat{q}_s(\hat{v}) = \varphi_s(\hat{v}) + \sum_{i=1}^n \min_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v}) (t - \hat{v}_i). \quad (4)$$

Then $\hat{q}_s(\hat{v}) \leq q_s(\lambda, \mu) \leq L_s(W_s)$, and each term of the sum is available in closed form:

$$\min_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v}) (t - \hat{v}_i) = \begin{cases} \partial_i \varphi_s(\hat{v}) (l_i - \hat{v}_i), & \partial_i \varphi_s(\hat{v}) > 0, \\ \partial_i \varphi_s(\hat{v}) (u_i - \hat{v}_i), & \partial_i \varphi_s(\hat{v}) < 0, \\ 0, & \partial_i \varphi_s(\hat{v}) = 0. \end{cases} \quad (5)$$

Proof. Convexity and differentiability at $\hat{v}$ give the gradient inequality

$$\varphi_s(v) \geq \varphi_s(\hat{v}) + \nabla \varphi_s(\hat{v})^\top (v - \hat{v}) \quad \text{for all } v.$$

Minimising both sides over $v \in B$ preserves the inequality, and the right-hand side is separable across coordinates because B is a box, which yields (4). Each one-dimensional problem minimises a linear function over an interval, so the minimum is attained at an endpoint, giving (5). The final inequality is Lemma 1. $\square$

Corollary 3. $\hat{q}_s(\hat{v}) \leq L_s(W_s)$ for any $\hat{v}$ at which φ_s is differentiable, any $\lambda \geq 0$ and any μ . In particular $\hat{v}$ need not be optimal, and need not even be feasible for (1).

Corollary 3 is the whole point. The certificate consumes the iterate the solver happened to stop at and the multipliers it happened to report, and returns a valid bound regardless. Computationally it costs one gradient evaluation and a loop over the variables — no second solve.

Remark 4 (Robustness). Because Corollary 3 holds for arbitrary $\lambda \geq 0$ and μ , an error in the sign convention used to recover the multipliers, a stale multiplier, or a multiplier clipped to zero can only make $\hat{q}_s$ smaller. Such an error makes the bound loose, never invalid. Convexity enjoys no such protection; see Section 5.

4 Exactness at a KKT point

The correction term in (4) is not merely small in practice: it vanishes identically at an exact KKT point, so the certificate degrades gracefully and costs nothing when the solve is good.

Proposition 5 (The correction vanishes). *Let $\hat{v}$ satisfy the KKT conditions of (1) with multipliers $\lambda \geq 0$ for $g_s \leq 0$, μ for $h_s = 0$, and $z_L, z_U \geq 0$ for the bounds $l - v \leq 0$ and $v - u \leq 0$. Then every term of the sum in (4) is zero, and if in addition φ_s is convex then $\hat{q}_s(\hat{v}) = L_s(W_s)$.*

Proof. Stationarity of the full Lagrangian, which is φ_s augmented with the bound terms, reads

$$\nabla \varphi_s(\hat{v}) = z_L - z_U. \quad (6)$$

Fix a coordinate i and consider the three possible positions of $\hat{v}_i$.

If $l_i < \hat{v}_i < u_i$, complementarity gives $z_{L,i} = z_{U,i} = 0$, so $\partial_i \varphi_s(\hat{v}) = 0$ by (6) and the term is 0 by (5).

If $\hat{v}_i = l_i$, complementarity gives $z_{U,i} = 0$, so $\partial_i \varphi_s(\hat{v}) = z_{L,i} \geq 0$. When the derivative is strictly positive (5) selects $t = l_i = \hat{v}_i$, and when it is zero the term is zero outright; either way the term is 0.

If $\hat{v}_i = u_i$, symmetrically $z_{L,i} = 0$ and $\partial_i \varphi_s(\hat{v}) = -z_{U,i} \leq 0$, so (5) selects $t = u_i = \hat{v}_i$ and the term is 0.

Hence $\hat{q}_s(\hat{v}) = \varphi_s(\hat{v})$. Complementarity $\lambda^\top g_s(\hat{v}) = 0$ and feasibility $h_s(\hat{v}) = 0$ reduce (2) to $\varphi_s(\hat{v}) = f_s(\hat{v}) + W_s^\top x(\hat{v})$. Under convexity the KKT conditions are sufficient for global optimality of (1), so that value is $L_s(W_s)$. $\square$

Proposition 5 has a practical corollary worth stating plainly: the correction term *measures its own inexactness*. It is zero when the solve is exact and grows as the solve degrades, so the reported bound loosens smoothly rather than becoming wrong.

5 Canonical form, and where convexity is easy to get backwards

Theorem 2 requires φ_s convex, and by (2) that requires each component of g_s to be convex and each component of h_s to be affine, since $\lambda \geq 0$ but μ is free in sign.

The subtlety is that g_s is the constraint in *canonical* form $g_s(v) \leq 0$, and putting a $\geq$ row into canonical form negates its body. Writing $b(v)$ for the constraint body as the user stated it:

as written	canonical form	requirement on b
$b(v) \leq \beta$	$g = b - \beta$	b convex
$b(v) \geq \alpha$	$g = \alpha - b$	b concave
$\alpha \leq b(v) \leq \beta$	both rows	b affine
$b(v) = \beta$	$h = b - \beta$	b affine

So $x^2 \leq 4$ is admissible and $x^2 \geq 1$ is not, though both are written with a convex body — and indeed the feasible set of the second is not convex. This is the one place in the construction where an entirely ordinary-looking model silently leaves the hypotheses, and the consequence is not a loose bound but an invalid one. A worked instance: for

$$\min x \quad \text{s.t.} \quad x^2 \geq 1, \quad x \in [0, 1.5],$$

whose optimum is 1, taking $\hat{v} = 0.5$ and $\lambda = 1$ in (4) yields $\hat{q} = 1.25 > 1$.

Of the four rows above, three are decidable — affineness is a question about polynomial degree — and the implementation enforces them. Convexity or concavity of a one-sided nonlinear body is not decidable, and remains a user assertion.

6 Aggregating across scenarios

The per-scenario bounds are combined by the usual probability-weighted sum, and the condition under which that sum is itself a bound deserves care.

Proposition 6 (Aggregation). *Let OPT be the optimal value of the original stochastic program. If the weights satisfy $\sum_s p_s W_s = 0$, then $\sum_s p_s L_s(W_s) \leq \text{OPT}$.*

Proof. Let x^* together with the scenario recourse decisions be optimal for the original problem, so that $\text{OPT} = \sum_s p_s f_s(v_s^*)$ where v_s^* is the optimal decision in scenario s and $x(v_s^*) = x^*$ for every s by nonanticipativity. Each v_s^* is feasible for subproblem (1), so $L_s(W_s) \leq f_s(v_s^*) + W_s^\top x^*$. Multiplying by p_s and summing,

$$\sum_s p_s L_s(W_s) \leq \sum_s p_s f_s(v_s^*) + \left(\sum_s p_s W_s \right)^\top x^* = \text{OPT} + 0. \quad \square$$

Remark 7 (Weights must share a generation). The hypothesis $\sum_s p_s W_s = 0$ is a *joint* condition on the whole weight vector, and progressive hedging maintains it at each iteration. It is not preserved by mixing: if W'_s takes scenario s 's weights from one iteration and scenario t 's from another, then in general $\sum_s p_s W'_s \neq 0$, the proof above leaves the residual term $(\sum_s p_s W'_s)^\top x^*$ in place, and the resulting number is not a bound at all. Nothing downstream can detect this, which is why a scenario whose certificate is unavailable must make the whole expectation unavailable rather than contribute a stale value.

7 Practical consequences

Tightness is governed by the box. By (5) the shortfall of $\hat{q}_s$ below $\varphi_s(\hat{v})$ is

$$\sum_{i=1}^n |\partial_i \varphi_s(\hat{v})| \cdot (\text{distance from } \hat{v}_i \text{ to the far end of } [l_i, u_i]),$$

so the bound is only as tight as the variable bounds are. Gradient components at a converged solve are small but generically nonzero, so a box of width 10 costs little while a box of width 10^{10} can cost a substantial fraction of the objective. Tightening bounds before certifying — for instance by feasibility-based bounds tightening, which shrinks B without removing feasible points and therefore only raises $\hat{q}_s$ — is the single most effective lever on quality.

Unbounded variables. If $l_i = -\infty$ and $\partial_i \varphi_s(\hat{v}) > 0$, or $u_i = +\infty$ and $\partial_i \varphi_s(\hat{v}) < 0$, the corresponding term in (5) is $-\infty$ and no finite bound is available. Whether this occurs depends on the model and on how converged the solve is: at an exact KKT point the offending component is zero by Proposition 5, but away from one it generally is not. The implementation reports “no bound” in that case rather than $-\infty$, so that Remark 7 is respected.

Floating point. Nothing in Theorem 2 requires a tolerance. A small relative cushion, reporting $\hat{q}_s - \varepsilon(1 + |\hat{q}_s|)$, is nevertheless cheap insurance against last-bit effects in the gradient and the sum; it is hygiene, not part of the argument.

8 Correspondence with the implementation

object here	in the code
φ_s in (2)	<code>_lagrangian_expression</code>
$\hat{q}_s$ in (4)	<code>certified_lower_bound</code>
decidable hypotheses	<code>check_model_is_certifiable</code>
box tightening	<code>unbounded_variables</code>
Proposition 6 sum	<code>SPOpt.Ebound</code>
Remark 7 safeguard	a <code>None</code> bound, propagated collectively

The first four live in `mpisppy/utils/dual_certificate.py`; the spoke that drives them is `mpisppy/cylinders/ipopt_outer_bound.py`.